NoisePrivacyMap<L1Distance<RBig>, MultiDP> for ZExpFamily<1>

Michael Shoemate (@Shoeboxam)

This proof resides in **"contrib"** because it has not completed the vetting process.

This proves the privacy-map implementation for the discrete Laplace mechanism when the output measure is MultiDP.

1 Hoare Triple

Precondition

MI is L1Distance<RBig> and MO is MultiDP. The distribution scale is a non-negative rational.

Pseudocode

```
class ZExpFamily1:
    def noise_privacy_map(self, _metric, _measure):
        scale = self.scale
        if scale < RBig.ZERO:  #
            raise "scale must be non-negative"

        def privacy_map(d_in):
            if d_in < RBig.ZERO:  #
                raise "sensitivity must be non-negative"
            if d_in == RBig.ZERO:  #
                return (
                    PrivacyGuarantee()
                    .with_approxDP([(0.0, 0.0)])
                    .with_zCDP(0.0, 0.0)
                    .with_renyiDP(lambda alpha: 0.0, 0.0)
                )
            if scale == RBig.ZERO:  #
                raise "no finite privacy guarantee"

            epsilon = f64.inf_cast(d_in / scale)
            sensitivity = f64.inf_cast(d_in)
            scale_f = f64.inf_cast(scale)
            if not epsilon.is_finite() or not sensitivity.is_finite() or scale_f <= 0:
                raise "privacy parameters are not finite"

            rho = zcdp_discrete_laplace(epsilon, sensitivity, scale_f)
            curve = PrivacyGuarantee().with_approxDP([(epsilon, 0.0)]).with_zCDP(rho, 0.0)
            rdp = lambda alpha: (
                rdp_discrete_laplace(alpha, sensitivity, scale_f)
                .or_else(lambda: rdp_from_pureDP(alpha, epsilon))
                .or_else(lambda: epsilon)
            )
            return curve.with_renyiDP(rdp, 0.0)
```

```
34
35          return PrivacyMap.new_fallible(privacy_map)
```

Postcondition

Theorem 1.1. For every input distance accepted by the returned privacy map, the map returns a `PrivacyGuarantee` containing valid privacy representations for adding an independent discrete Laplace sample to each coordinate.

Proof. Lines 4 and 8 reject invalid parameters. With zero sensitivity, line 10 returns the identity guarantee. With positive sensitivity and zero scale, the mechanism has no finite guarantee and line 17 reports failure.

For positive rational sensitivity s and scale b, the discrete Laplace point-mass ratio calculation gives the pure-DP bound

$$\varepsilon = s/b.$$

The conservative casts in the pseudocode make the floating-point `epsilon` an upper bound on this value, so the $(\varepsilon, 0)$ point inserted into the guarantee is valid.

The implementation also computes the Harrison–Manurangsi zCDP bound using directed interval arithmetic. Therefore the returned ρ is no smaller than the exact zCDP bound. Finally, `rdp_discrete_laplace` evaluates the exact Rényi expression with directed intervals. If that evaluation fails, the implementation first uses the exact RDP value implied by pure DP and, if those numerics also fail, uses the elementary pure-DP bound $D_\alpha \leq \varepsilon$. Each result is an upper bound at every valid order. Each representation is therefore simultaneously valid, and querying or composing the resulting `PrivacyGuarantee` preserves conservativeness. □